

Artificial Intelligence Assignment 2

Project Overview

This project involves building an Artificial Intelligence (AI) to play the game Connect 4. The program allows a human player to play against the computer. The computer chooses its moves by predicting future board states using different search algorithms. The algorithms implemented for the AI are Minimax, Alpha-Beta Pruning, and Expected Minimax.

Choosing the Best Heuristic

We decided not to use a simple piece-counting heuristic because it does not understand the concept of "threats." Just having more pieces on the board does not mean you are winning in Connect 4. Instead, we designed a heuristic that rewards finding actual lines that could lead to a win, controlling the center of the board, and penalizing the opponent's setups.

This is considered the "best" heuristic because it uses an exponentially scaled weight system. By scaling the numbers exponentially instead of linearly (like 1, 2, 3), the heuristic creates a strict hierarchy of goals for the AI: **Win the Game > Threaten to Win > Develop Pieces > Positional Advantage**

Without exponential scaling, the AI could make terrible, weird mathematical tradeoffs. For example, if an "open three" was worth 50 points and a "win" was only worth 100 points, the AI might let the opponent win if it thought it could get three different "open threes" instead! Exponential scaling guarantees the AI never trades a win for a bunch of smaller, useless advantages.

How the Heuristic Calculates the Score

The heuristic evaluates the board by giving it a total mathematical score. It starts at a score of 0. The algorithm scans every possible 4-cell window on the entire board (horizontally, vertically, and diagonally).

For every window, it counts the pieces inside. If the pieces belong to the computer, it ADDS points to the total score. If the pieces belong to the human, it SUBTRACTS points from the total score. Finally, it adds points for every computer piece in the center column and subtracts points for human pieces in the center.

The final result is a single number. A high positive score means the computer has better positioning, and a negative score means the human has the advantage.

How Each Value Was Calculated

- **Center column control (9 points per piece):** The center column gets 9 points because most ways to get 4-in-a-row physically have to pass through it. This value is relatively low because it's meant to encourage good positioning early in the game. Filling the whole center column (6 pieces) gives 54 points, which slightly outweighs having a single "open two".
- **2 pieces and 2 empty spaces (45 points):** This represents an early threat (an "open two"). We need this score to outweigh simple center column control.
- **3 pieces and 1 empty space (220 points):** This represents a massive threat (an "open three") and is one move away from winning. Notice that 220 is about 5 times bigger than 45. We calculated this

so that getting a single "open three" is mathematically better than having four "open twos" ($45 * 4 = 180$). It teaches the AI to focus its pieces into one dangerous line rather than scattering them.

- **4 pieces in a row (12,000 points):** This acts as mathematical "infinity". Even if the board was magically filled with as many "open threes" as possible, the score would never mathematically reach 12,000. We calculated that this massive gap guarantees the AI will immediately claim a win (or block an opponent's win) unconditionally, making it impossible for the AI to get distracted by other shiny things on the board.

Real Example of the Heuristic Calculation

Imagine the board has only two points of interest to keep it simple:

Situation 1: The computer has a line of [Computer, Computer, Empty, Empty]. This is an "open two". The heuristic sees this window and ADDS points to the total score based on the weight (e.g., adding +90 points).

Situation 2: The human has a line of [Human, Human, Human, Empty] on the same board. This is an "open three" and the human is one move away from winning. The heuristic sees this and SUBTRACTS a chunk of points from the total score (e.g., subtracting -440 points).

Calculated Score: The final score comes out to $90 - 440 = -350$. Because the score is deeply negative, the computer instantly realizes the board state is very dangerous for it. The mathematical penalty forces it to drop its own piece in that Empty spot to block the human and prevent the total score from crashing.

Search Algorithms

1. Minimax

The Minimax algorithm looks ahead into the game by checking all possible moves up to a certain depth limit. It assumes that the AI will always pick the move that gives the highest score, and the human will always pick the move that gives the lowest score. The AI builds a tree of possibilities and works backwards to find the current best move. The downside is that it takes a long time because it evaluates every single possibility.

2. Alpha-Beta Pruning

This is an upgraded, faster version of Minimax. While checking future moves, if the AI finds a branch that is clearly worse than a move it already discovered, it stops searching down that branch. *Example:* If the AI already found a move choice that guarantees a score of +100, and starts checking a new branch where the human can immediately force a score of -500, the AI stops analyzing the rest of that new branch. It knows it will never choose it anyway. This "pruning" saves a lot of time and lets the AI think deeper.

3. Expected Minimax

In this mode, the game introduces probability and chance instead of perfect moves. When the AI tries to drop a piece in a column, it is not guaranteed to land exactly there. For example, there might be a 60% chance it lands in the chosen column, a 20% chance it slips into the column to the left, and a 20% chance it slips to the right.

Instead of just taking the absolute maximum score like standard Minimax, the algorithm calculates the "expected score". It multiplies the score of each possible final position by the percentage chance of it happening, and adds them up.

Algorithm Comparison and Benchmarks

To see the difference in how the three algorithms perform, we ran tests to measure the **Time Taken (ms)** and the **Nodes Expanded** at different depths (K values). The test was run from the second move of the game (after the human player places a piece in the center column).

Depth (K)	Algorithm	Nodes Expanded	Time Taken (ms)
K = 1	Minimax	8	~0.35
K = 2	Minimax	57	~2.33
K = 3	Minimax	400	~15.91
K = 4	Minimax	2801	~122.04
K = 5	Minimax	19608	~693.01
K = 1	Alpha-Beta	8	~0.34
K = 2	Alpha-Beta	27	~0.84
K = 3	Alpha-Beta	101	~3.31
K = 4	Alpha-Beta	387	~11.63
K = 5	Alpha-Beta	1341	~45.21
K = 1	Expected	20	~0.90
K = 2	Expected	153	~5.82
K = 3	Expected	2680	~107.87
K = 4	Expected	20369	~728.26
K = 5	Expected	356460	~14056.08 (14s)

Conclusion from Data: As seen from the data, Minimax explodes in time as the depth increases because it checks every single node. Alpha-Beta is significantly faster and expands far fewer nodes due to pruning (at depth 5, Alpha-Beta only expanded 1,341 nodes compared to Minimax's 19,608). Expected Minimax takes the longest amount of time by far (nearly 14 seconds for depth 5) because it has to simulate probability branches (left slip, right slip) for every single move on top of normal moves.

Sample AI Execution Trees

Here are real examples of the search trees generated by the different AI algorithms during gameplay.

1. Plain Minimax (Depth 1)

In strict Minimax, the AI simply iterates over every possible column to maximize its own score. At Depth 1, there are 7 regular branches (columns).

```
-----
-
[SEARCH] AI search start | algorithm=minimax | depth=1 | nodes=0
-----
-
MAX          depth=1 alpha=-inf beta=inf
  LEAF        depth=0 heuristic=0.00
-> column=3 -> score=0.00
  LEAF        depth=0 heuristic=-9.00
-> column=2 -> score=-9.00
  LEAF        depth=0 heuristic=-9.00
-> column=4 -> score=-9.00
  LEAF        depth=0 heuristic=-9.00
-> column=1 -> score=-9.00
  LEAF        depth=0 heuristic=-9.00
-> column=5 -> score=-9.00
  LEAF        depth=0 heuristic=-9.00
-> column=0 -> score=-9.00
  LEAF        depth=0 heuristic=-9.00
-> column=6 -> score=-9.00
MAX-CHOICE   column=3 score=0.00

-----
-
[SEARCH] AI search end | move=3 | score=0.00 | nodes=8 | time=0.40 ms
-----
-
```

2. Alpha-Beta Pruning (Depth 2)

Notice how Alpha-Beta saves time inside **MIN** nodes. When the AI analyzes **column=3** looking ahead to depth 2, it calculates **-108.00**, but soon after when testing **column=4** it notices a terrible result and triggers **PRUNE** to exit the branch early, skipping unnecessary checks.

```
-----
-
[SEARCH] AI search start | algorithm=alpha-beta | depth=2 | nodes=0
-----
-
MAX          depth=2 alpha=-inf beta=inf
  MIN        depth=1 alpha=-inf beta=inf
    LEAF      depth=0 heuristic=-9.00
-> column=3 -> score=-9.00
    LEAF      depth=0 heuristic=-270.00
-> column=2 -> score=-270.00
    LEAF      depth=0 heuristic=-270.00
-> column=4 -> score=-270.00
```

```

LEAF          depth=0 heuristic=-180.00
-> column=1 -> score=-180.00
LEAF          depth=0 heuristic=-180.00
-> column=5 -> score=-180.00
LEAF          depth=0 heuristic=-90.00
-> column=0 -> score=-90.00
LEAF          depth=0 heuristic=-90.00
-> column=6 -> score=-90.00
MIN-CHOICE    column=2 score=-270.00
-> column=3 -> score=-270.00

MIN           depth=1 alpha=-270.00 beta=inf
  LEAF        depth=0 heuristic=-108.00
  -> column=3 -> score=-108.00
  LEAF        depth=0 heuristic=-99.00
  -> column=2 -> score=-99.00
  LEAF        depth=0 heuristic=-99.00
  -> column=4 -> score=-99.00
  LEAF        depth=0 heuristic=-9.00
  -> column=1 -> score=-9.00
  LEAF        depth=0 heuristic=-99.00
  -> column=5 -> score=-99.00
  LEAF        depth=0 heuristic=-9.00
  -> column=0 -> score=-9.00
  LEAF        depth=0 heuristic=-99.00
  -> column=6 -> score=-99.00
MIN-CHOICE    column=3 score=-108.00
-> column=2 -> score=-108.00

MIN           depth=1 alpha=-108.00 beta=inf
  LEAF        depth=0 heuristic=-108.00
  -> column=3 -> score=-108.00
  PRUNE       alpha=-108.00 beta=-108.00
MIN-CHOICE    column=3 score=-108.00
-> column=4 -> score=-108.00

```

```

MAX-CHOICE    column=2 score=-108.00

```

```

-----
-
[SEARCH] AI search end | move=2 | score=-108.00 | nodes=27 | time=1.02 ms
-----
-

```

3. Expected Minimax (Depth 1)

Here, the trace structure completely changes. Instead of normal nodes, Expected Minimax creates **CHANCE** branches. When evaluating **column=3** at Depth 1, it realizes there are 3 possible outcomes based on probabilities (dropping right down, sliding left, sliding right). It multiplies them together to output a raw mathematical **expected** chance (-3.60).

```

-----
-
[SEARCH] AI search start | algorithm=expected | depth=1 | nodes=0
-----
-
EXPECT-MAX    depth=1
  CHANCE      column=3 outcomes=3
  CHANCE-RESULT column=3 expected=-3.60
  -> column=3 -> expected_score=-3.60

  CHANCE      column=2 outcomes=3
  CHANCE-RESULT column=2 expected=-7.20
  -> column=2 -> expected_score=-7.20

  CHANCE      column=4 outcomes=3
  CHANCE-RESULT column=4 expected=-7.20
  -> column=4 -> expected_score=-7.20

  CHANCE      column=1 outcomes=3
  CHANCE-RESULT column=1 expected=-9.00
  -> column=1 -> expected_score=-9.00

  CHANCE      column=5 outcomes=3
  CHANCE-RESULT column=5 expected=-9.00
  -> column=5 -> expected_score=-9.00

  CHANCE      column=0 outcomes=2
  CHANCE-RESULT column=0 expected=-9.00
  -> column=0 -> expected_score=-9.00

  CHANCE      column=6 outcomes=2
  CHANCE-RESULT column=6 expected=-9.00
  -> column=6 -> expected_score=-9.00
EXPECT-CHOICE column=3 score=-3.60

-----
-
[SEARCH] AI search end | move=3 | score=-3.60 | nodes=20 | time=1.33 ms
-----
-

```

Data Structures Used

- **2D List (`List[List[int]]`):** The `Connect4Board` itself is represented using a 2D list array for the grid because it is fast enough for checking sliding 4-cell windows sequentially, and requires no complicated external libraries.
- **Data Classes:** Python `dataclass` is heavily used throughout the project (e.g. `HeuristicWeights`, `MoveRecord`, `SearchStats`) to logically group related information like search metadata without having to manually construct bulky class initializers.

- **Trees (Implicit):** The algorithm builds an implicit search tree in memory. Python's native call stack builds out the branches structure recursively during search, relying on returning multiple tuples of `(score, column)`.

Assumptions and Clarifications

- **Zero-Sum Game Presumption:** The heuristic intrinsically assumes a perfect zero-sum game setup (if the AI gains an advantage, it is equal to the human losing an advantage).
- **Overlapping Connect-4s:** It is assumed per traditional rules that while the game checks for all connecting windows, multiple lines on the board are scored concurrently.
- **Probabilistic Spread (Expected Minimax):** It is calculated assuming only immediate-neighbor placement issues (i.e. left/right columns on drop). The assignment probability relies on the random chance affecting the current drop only, not an ongoing sequence of unpredictable gravity anomalies.